

Unit 4

Content

- Tuples
- Dictionaries
- Lists
- Sets
- Queues
- strings and serialization
- regular expressions
- Iterator comprehensions
- Generators

Tuple

- Creating Tuple
- Traversing through tuple
- Tuple methods

Def: In Python, a tuple is a built-in data type that allows you to create *immutable sequences* of values. The values or items in a tuple can be of any type.

Characteristics

The characteristics of a Python tuple are:

1. Tuples are ordered, indexed collections of data. Similar to string indices, the first value in the tuple will have the index [0], the second value [1], and so on.
2. Tuples can store duplicate values.
3. Once data is assigned to a tuple, the values cannot be changed.
4. Tuples allow you to store several data items in one variable. You can choose to store only one kind of data in a tuple or mix it up as needed.

How to Create and Use Tuples?

Two methods

- Using ()
- Using the **tuple()** Constructor

Access the items in an existing tuple

Syntax:

```
tuple_object[index]
```

Retrieving Multiple Items From a Tuple: Slicing

Like other Python sequences, tuples allow you to extract a portion or slice of their content with a slicing operation.

Syntax: **tuple_object[start:stop:step]**

Index	Description	Default Value
start	Specifies the index at which you want to start the slicing. The item at this index is included in the final slice.	0
stop	Specifies the index at which you want the slicing to stop extracting items. The item at this index isn't included in the final slice.	<code>len(tuple_object)</code>
step	Provides an integer value representing how many items the slicing will jump through on each step. If step is greater than 1, then jumped items won't be in the resulting slice.	

Tuple Methods

`count()` :Returns the number of times a specified value occurs in a tuple

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
```

```
x = thistuple.count(5)
```

```
print(x)
```

`index()`: Searches the tuple for a specified value and returns the position of where it was found

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
```

```
x = thistuple.index(8)
```

```
print(x)
```

Set

- Creating set
- Traversing through set
- Set Operations
- User input for set

Creating Set

A Set in Python programming is an unordered collection data type that is iterable, mutable and has no duplicate elements.

- Using { }
- Using Set() constructor
- frozenset() : frozenset is immutable. You can perform non-modifying operations on a frozenset:

Traversing through set

Accessing Element through index:

E.g.

```
my_set = {'london', 'new york', 'seattle', 'sydney', 'chicago'}
```

```
for item in my_set:
```

```
    print(item)
```

Python Set add() Method

- The add() method adds an element to the set.
- If the element already exists, the add() method does not add the element.

Syntax

```
set.add(elmnt)
```

E.g.

```
fruits = {"Mango", "banana", "cherry"}
```

```
fruits.add("apple")
```

```
print(fruits)
```

Python Set clear() Method

The clear() method removes all elements in a set.

Syntax

```
set.clear()
```

E.g.

```
fruits = {"apple", "banana", "cherry"}
```

```
fruits.clear()
```

```
print(fruits)
```

Python Set copy() Method

The copy() method copies the set.

Syntax

```
set.copy()
```

```
fruits = {"apple", "banana", "cherry"}
```

```
x = fruits.copy()
```

```
print(x)
```

Python Set difference() Method

- The difference() method returns a set that contains the difference between two sets.
- Meaning: The returned set contains items that exist only in the first set, and not in both sets.
- As a shortcut, you can use the “-” operator instead.

Syntax

`set.difference(set1, set2 ... etc.)`

Or

`set - set1 - set2 .... etc.`

Python Set difference() Method Example

```
a = {"apple", "banana", "cherry"}
```

```
b = {"google", "microsoft", "apple"}
```

```
myset = a - b
```

```
print(myset)
```

Example

```
a = {"apple", "banana", "cherry"}  
b = {"google", "microsoft", "apple"}  
c = {"cherry", "micra", "bluebird"}
```

```
myset = a.difference(b, c)
```

```
print(myset)
```

```
a = {"apple", "banana", "cherry"}  
b = {"google", "microsoft", "apple"}  
c = {"cherry", "micra", "bluebird"}
```

```
myset = a - b - c
```

```
print(myset)
```

Python Set difference_update() Method

- The difference_update() method removes the items that exist in both sets.
- The difference_update() method is different from the difference() method, because the difference() method returns a new set, without the unwanted items, and the difference_update() method removes the unwanted items from the original set.

Example 1

```
x = {"apple", "banana", "cherry"}
```

```
y = {"google", "microsoft", "apple"}
```

```
x.difference_update(y)
```

```
print(x)
```

Example 2

```
a = {"apple", "banana", "cherry"}
```

```
b = {"google", "microsoft", "apple"}
```

```
a -= b
```

```
print(a)
```

Example 3

```
a = {"apple", "banana", "cherry"}
```

```
b = {"google", "microsoft", "apple"}
```

```
c = {"cherry", "micra", "bluebird"}
```

```
a -= b | c
```

```
print(myset)
```

Python Set intersection() Method

- The intersection() method returns a set that contains the similarity between two or more sets.
- Meaning: The returned set contains only items that exist in both sets, or in all sets if the comparison is done with more than two sets.
- As a shortcut, you can use the & operator.

Example 1

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}  
z = x & y  
print(z)
```

```
x = {"a", "b", "c"}  
y = {"c", "d", "e"}  
z = {"f", "g", "c"}  
result = x.intersection(y, z)  
print(result)
```

Python Set union() Method

- The union() method returns a set that contains all items from the original set, and all items from the specified set(s).
- You can specify as many sets you want, separated by commas.
- It does not have to be a set, it can be any iterable object.
- If an item is present in more than one set, the result will contain only one appearance of this item.
- As a shortcut, you can use the | operator.

Example

```
x = {"a", "b", "c"}  
y = {"f", "d", "a"}  
z = {"c", "d", "e"}  
  
result = x.union(y, z)  
  
print(result)
```

```
x = {"a", "b", "c"}  
y = {"c", "d", "e"}  
z = {"f", "g", "c"}  
  
result = x | y | z  
  
print(result)
```

User input for set

```
str1 = input("please enter a string:")
```

```
str_set = set(str1)
```

```
print(str_set)
```

Dictionary

- Creating Dictionary
- Traversing through Dictionary
- Dictionary Methods
- User input for Dictionary

About Dictionary

- The dictionary is an unordered collection that contains *key:value* pairs separated by commas inside curly brackets.
- Dictionaries in Python is a *data structure*.
- Dictionaries are optimized to retrieve values when the key is known.

E.g.

```
SubjectTeachers = {"Python":"P. M. Kamble", "Prog. C":"Shilpa", "COA":"Kagale M."}
```

```
print(type(capitals))
```

Creating Dictionary

1) Using { }

Python Dictionary Syntax

```
dict_var = {key1 : value1, key2 : value2, .....}
```

2) The dict() Constructor

```
studentdict = dict(name = "Rahul", age = 20, country = "India")
```

```
print(studentdict)
```

Nested Dictionary

```
Dict = {'20UGMCS01': 'Abdul Nafay', '22UGMCS31': 'Yogit',  
        '22UGMCS19': {'Python': 89, 'COA': 99, 'Maths': 78}}  
  
print(Dict)
```

Traversing through Dictionary

- Print all **key** names in the dictionary, one by one:

```
thisdict ={"brand": "KIA", "model": "Seltos", "year": 2021}
```

```
for x in thisdict:
```

```
    print(x)
```

- Print all **values** in the dictionary, one by one:

```
for x in thisdict:
```

```
    print(thisdict[x])
```

- Loop through both **keys and values**, by using the items() function:

```
for x, y in thisdict.items():
```

```
    print(x, y)
```

Python Dictionary Methods

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a <u>list containing a tuple</u> for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

fromkeys()

```
seq = {'a', 'b', 'c', 'd', 'e'}
```

```
res_dict = dict.fromkeys(seq)
```

```
print("The newly created dict with None values : " + str(res_dict))
```

```
res_dict2 = dict.fromkeys(seq, 1)
```

```
print("The newly created dict with 1 as value : " + str(res_dict2))
```

User input for Dictionary

```
user_dict = {}  
  
num_entries = int(input("Enter the number of entries you want to add: "))  
  
for i in range(num_entries):  
    key = input("Enter key: ")  
    value = input("Enter value: ")  
    user_dict[key] = value  
  
print("Dictionary after adding user input:", user_dict)
```